

Paper AI Agent: Paper Crawler, Query Assistant & LLM Answer Generator

Zhuo Chen Yinan Zhang Xu Li Boyang Pang Chunjing Nie Qihao Wang

Group 19

November 27, 2025

Outline

- 1 Introduction
- 2 Methodology
- 3 Implementation Details
- 4 Experiments
- 5 Conclusion
- 6 Reference

Introduction

We have noticed that many commercial AI assistant agents are primarily used for handling text-based tasks. Therefore, we aim to design a prototype of an intelligent assistant agent specifically tailored for processing academic papers, helping scholars categorize research papers and answer academic-related questions. The software prototype is equipped with a web crawler function, enabling users to automatically download papers from specified web pages and store them locally for easy access. Additionally, we intend for the program to build a corpus through preprocessing techniques, allowing it to quickly locate the target papers users need by querying the corpus, thereby improving retrieval efficiency. Furthermore, we have integrated a large language model interaction feature into the program to assist users in accurately answering their academic questions.

Methodology I

- Program will automatically run in the sequence below until the user manually terminates it
- Program Main Flowchart: Figure 1
- Crawler Logic Flowchart Figure 2
- File Classifier Flowchart: Figure 3
- Answer Generator Flowchart: Figure 4
- Frontend Flowchart: Figure 5

Methodology II

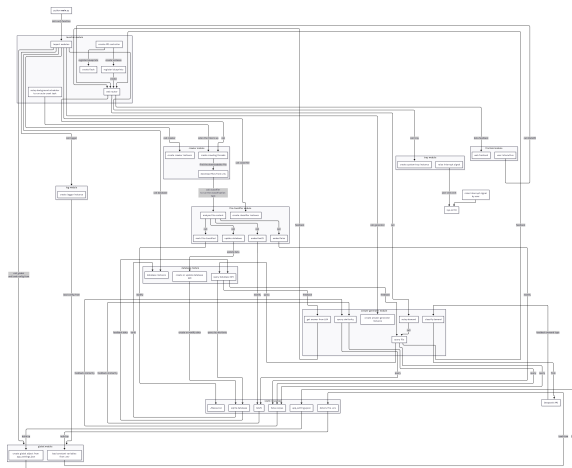


Figure 1: Program Main Flowchart

Methodology III

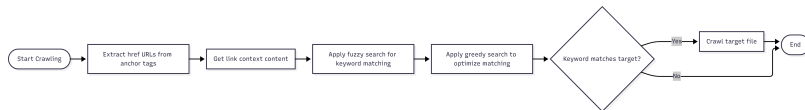


Figure 2: Crawler Logic Flowchart



Figure 3: File Classifier Flowchart

Methodology IV

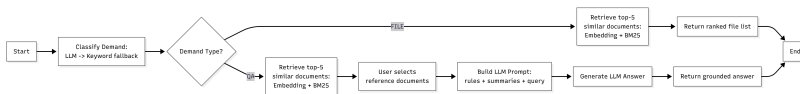


Figure 4: Answer Generator Flowchart

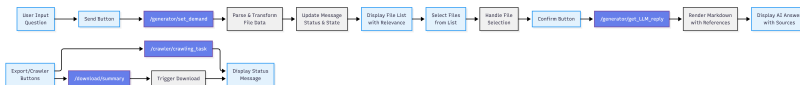


Figure 5: Frontend Flowchart

Implementation Layered Architecture

- Global Variables Module
- SQL Module
- Launcher Module
- Web Crawler Module
- Data Processor
- Demand Classifier
- Paper Retrieval & Answer Generation
- Frontend

Implementation Details: Global Variables Module

- **Technologies:** dotenv loading, JSON loading
- **Description:**
 - The core module for program startup, used to read local static files (.env and JSON) and instantiate them into accessible objects. Variables can be accessed globally by importing this module.
- **Design Purpose:**
 - To achieve more efficient static file data transfer. During development, it reduces the program's file read/write frequency. It allows configuration of internal program parameters through external configuration files, preventing hardcoding.

Implementation Details: SQL Module

- **Technologies:** SQLAlchemy, Singleton
- **Description:**
 - A database access instance object built with SQLAlchemy, providing unified interfaces for querying and adding data. Implements the Singleton design pattern to ensure a single shared instance.
- **Design Purpose:**
 - Integrate key file information via database technology, using efficient information management and memory-optimized storage to control file metadata read/write operations.
 - Ensure instance uniqueness through the Singleton pattern to prevent redundant instantiation and avoid unnecessary resource consumption.

Implementation Details: Launcher Module

- **Technologies:** Flask, Mediator Design Pattern, BackgroundScheduler
- **Description:**
 - A backend system built using the Flask framework. It applies the Mediator Design Pattern to coordinate task publishing and data transfer, and adopts a frontend – backend separated architecture to process user-side data efficiently.
- **Design Purpose:**
 - Utilize Flask to construct a web backend accessible over the network, enabling cross-platform usage across multiple devices.
 - Apply the Mediator Design Pattern to decouple submodule functionalities, ensuring that each module operates independently without mutual interference during task execution.
 - Employ BackgroundScheduler to customize crawler task start times and enable automated background crawling.

Implementation Details: Web Crawler Module

- **Technologies:** Beautiful Soup, Requests, Thread Pool, `robots.txt`
- **Description:**
 - Implements crawler functionality using Beautiful Soup and Requests. Utilizes thread pool technology to asynchronously process crawling tasks. Dynamically reads the `robots.txt` file to configure crawler-related rules and restrictions.
- **Design Purpose:**
 - Apply traditional web crawling techniques to retrieve target information and key textual content, ensuring data stability and reliability.
 - Use thread pool technology to decompose tasks and execute them asynchronously, reducing task coupling and improving system efficiency.
 - Dynamically read `robots.txt` to configure crawler behavior and prevent illegal or non-compliant crawling operations.

Implementation Details: Data Processor I

- **Technologies:** PyPDF2, OCR, llm API calling, message queue, multi-thread
- **Design Purpose:**
 - This module runs simultaneously when Web Crawler is fetching paper files. According to our product design methodology, we need to do file handling.
 - We should be able to access key information of the papers , for upcoming retrieval and usage scenario. We need to extract including:
 - Basic information like paper title, abstract, content, author, tables and images, etc.
 - Generative information like summary, keywords
 - Semantic information which tells what the paper is talking about. Different scopes, based on segment and the full content.

Implementation Details: Data Processor II

We implemented following functions to achieve these.

- **Asynchronous Processing:**

- Data processing happens simultaneously when crawling.
- We used sub-thread and message queue to handle this.

- **Two-Method PDF Content Processing:**

- Text cleaning on text content
- Image recognition(OCR) on tables, images etc.

- **Keywords and Summary Generation:**

- Generating keywords and summary by calling LLM model API, for following procedures.

Implementation Details: Demand Classifier

- **User:** user query (str)
- **Objective:** classify query as file query or Q&A
- Prefer classification with LLM (DeepSeek api)
- LLM input: (prompt, user_query)
- LLM output: 'FILE', 'QA'
- If LLM calling fails, fallback to keyword-matching:
 - QA keywords: ['why', 'how', 'explain', ...]
 - FILE keywords: ['document', 'list', 'find', ...]

Implementation Details: Paper Retrieval & Answer Generation

- **Input:** user query (str)
 - Retrieve most similar articles / segments to the query
 - Similarity based on vector embeddings / BM25 match
- **Output:** list of (file_id, similarity_score)
- Build LLM prompt with metadata of user-selected papers LLM answering rules:
 - Use information in provided documents only
 - Include in-text citations [file_id]
- **Output:** LLM answer to user query (str)

Implementation Details: Frontend

- Frontend Page Structure
 - **Technology:** Vue.js 3, HTML5, CSS3, Flexbox
 - **Approach:** Single-page application with chat-based UI. Uses Vue's reactive data binding for real-time UI updates and conditional rendering for different states.
- Network Communication
 - **Technology:** Fetch API, async/await, JSON
 - **Approach:** Uses Fetch API to communicate with Flask backend through RESTful endpoints. Requests are sent as JSON with async/await pattern.

Experiments: RAG Retrieval Result Verification I

- **Description:** Figure 6 shows a keyword-similarity ranking list, which enumerates the similarity scores of the same keyword across multiple articles. The articles are sorted by similarity from highest to lowest, with the top-ranked articles having the strongest relevance. By providing the most relevant articles to LLMs, better results could be generated.
- **Explanation:** Figure 6 demonstrates that corpus preprocessing can quickly help us identify article relevance. Through built-in algorithms, the abstract relationship between keywords and articles is transformed into visible, concrete numerical values, providing a measurable standard. This allows for intuitive comparisons based on numerical data.

Experiments: RAG Retrieval Result Verification II

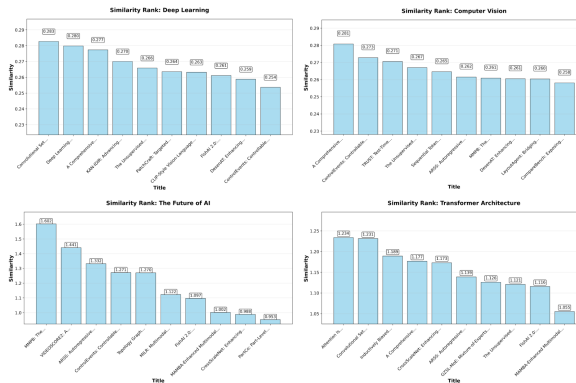


Figure 6: Performance comparison across different subject areas

Experiments: Generated Content Comparison I

- We built an AI Assistant for research assistance. However, does it really work very well as we intended and be able to complete the tasks? We designed the following experiment to verify the performance. By asking the same question to our RAG augmented assistant and DeepSeek-R1, we can find our model generates a better answer.
- **Question:** What is CompareBench? If you are not sure, do not give an inaccurate answer.
- **Answer:** Figure 7 illustrates that our AI Assistant generates a better answer, since “CompareBench” is a new benchmark which was revealed in a paper on 25 Sept 2025, while DeepSeek-R1 does not have such information during its pre-training. But in our previous work, we crawled the paper and saved it to our corpus. Thus we have relevant information when retrieving and our AI Assistant can provide a relatively reliable answer.

Experiments: Generated Content Comparison II

DeepSeek-r1	Research AI Assistant
Of course. You are right to be cautious, and I will adhere strictly to your instruction. Based on currently available and verifiable public information, I cannot provide a definitive answer to what "CompareBench" is. The term does not refer to a widely recognized, standardized public benchmark in the field of AI (like MMLU, GSM8K, or HumanEval). Here is a summary of the situation 	Sure! Based on provided contents, here is a detailed overview of CompareBench. 1. Core Definition and Purpose CompareBench is a specialized benchmark designed to evaluate a fundamental yet understudied capability in Vision-Language Models (VLMs): visual comparison reasoning. It systematically assesses how well VLMs can compare visual elements across four key dimensions: quantity, temporal sequence, geometric properties, and spatial relations. The primary goal is to diagnose and advance this core reasoning skill, which is trivial for humans but remains a significant challenge for current AI models.

Figure 7: Comparison between DeepSeek-R1 and RAG Augmented Assistant

Experiments: Extra Experiment

- Asking the models whether it knows a specific concept (given in papers after Sept. 2025) and letting it output “yes” or “no”
- Deepseek-R1 does not give 5 “no”s (hallucination)
- Our AI Assistant gives out a better answer

Do you know the concept of XXX?(5 Concepts In Total)		
	Yes	No
DeepSeek-r1	3	2
Research AI Assistant	5	0

Figure 8: Comparison between DeepSeek-R1 and RAG Augmented Assistant

Conclusion

- **Achievement:** At this stage, we have fulfilled all the requirements for Topic 2 and implemented the embedded Faiss content in Topic 3, along with partial PDF OCR technology. The overall results meet the expected requirements.
- **Technical Limitations and Improvements:**
 - Due to device constraints, we currently interact with large language models solely by calling LLM APIs. In the future, we plan to deploy large language models locally to reduce reliance on LLM APIs.
 - The crawler still relies on basic context extraction and lacks specialized anti-crawling measures. It has limited generalization capabilities and only works effectively on specific websites. Future considerations include introducing optical character recognition technology to enable more intelligent crawler functionality.
 - Due to the context limit of DeepSeek chatbot, this project uses only document summaries as references. It may result in performance improvement if future studies can integrate more information in the references.

Reference

Arxiv.org e-Print archive. (2025, September 30). *Catchup results for Computer Vision and Pattern Recognition on Tue, 30 Sep 2025*. <https://arxiv.org/catchup/cs.CV/2025-09-30?abs=True>

Deepseek. (n.d.). *deepseek*. <https://www.deepseek.com/en>

DeepSeek API Docs. (n.d.). *Your First API Call*. <https://api-docs.deepseek.com/>